

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import zipfile
import os

# Assuming the uploaded file is named 'archive.zip'
zip_file_path = '/content/drive/MyDrive/dataset/archive.zip'

# Check if the zip file exists
if os.path.exists(zip_file_path):
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
        zip_ref.extractall('.') # Extract all contents to the current directory
    print(f"Successfully unzipped '{zip_file_path}'")
    print("Extracted files:")
    for filename in zip_ref.namelist():
        print(f"- {filename}")
else:
    print(f"Error: Zip file '{zip_file_path}' not found.")
```

Successfully unzipped '/content/drive/MyDrive/dataset/archive.zip'
 Extracted files:
 - faker_employee.csv

```
!pip install pyspark
```

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-package
 Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-package

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("CSV_Reader").getOrCreate()
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("/content/faker_employee.csv")

df.printSchema()
```

```
root
|-- Employee ID: string (nullable = true)
|-- First Name: string (nullable = true)
|-- Last Name: string (nullable = true)
|-- Department: string (nullable = true)
|-- Salary: integer (nullable = true)
```

```
-- Joining Date: date (nullable = true)
-- Email ID: string (nullable = true)
-- Address: string (nullable = true)
```

```
from pyspark.sql.functions import col, sum as spark_sum, rank
from pyspark.sql.window import Window
import csv
from io import StringIO
```

```
df_clean = df.na.drop(subset=["Salary", "Department"])
print("Total rows:", df.count())
print("Clean rows:", df_clean.count())
```

```
# --- filter: salary > 70000 ---
high_salary = df_clean.filter(col("Salary") > 70000)
high_salary.show(5)
```

```
Total rows: 2000000
Clean rows: 1000000
```

```
+-----+-----+-----+-----+-----+-----+-----+
|Employee ID|First Name|Last Name|      Department|Salary|Joining Date|
+-----+-----+-----+-----+-----+-----+-----+
|         4|  Brittney|  Morgan|      Sales| 72174| 2022-10-03|wcervante
|        15|   Andrew|   Joyce|Human Resources| 74864| 2023-03-18| katie91@
|        16| Stephanie|   Smith|Customer Service| 72876| 2023-05-22| john34@
|        19|    Karen| Mitchell|   Operations| 75018| 2023-09-27|ramirezkr
|        24|  Michael| Figueroa|      Legal| 76447| 2022-12-07| sriley@
+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
# --- null check ---
df.select([spark_sum(col(c).isNull().cast("int")).alias(c) for c in df.columns])
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
|Employee ID|First Name|Last Name|Department| Salary|Joining Date|Email ID|Address
+-----+-----+-----+-----+-----+-----+-----+-----+
|          0|   107096| 1000000| 1000000|1000000| 1000000| 1000000|1000000
+-----+-----+-----+-----+-----+-----+-----+-----+
```

```
# --- filter: salary > 70000 ---
high_salary = df_clean.filter(col("Salary") > 70000)
high_salary.show(5)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|Employee ID|First Name|Last Name|      Department|Salary|Joining Date|
```

```
+-----+-----+-----+-----+
|      4| Brittney| Morgan| Sales| 72174| 2022-10-03|wcervante
|     15| Andrew| Joyce| Human Resources| 74864| 2023-03-18| katie91@
|     16| Stephanie| Smith| Customer Service| 72876| 2023-05-22| john34@
|     19| Karen| Mitchell| Operations| 75018| 2023-09-27|ramirezkr
|     24| Michael| Figueroa| Legal| 76447| 2022-12-07| sriley@
+-----+-----+-----+-----+
only showing top 5 rows
```

```
# --- join ---
dept_data = [
    ("Sales", 500000),
    ("Legal", 300000),
    ("R&D", 800000),
    ("Training and Development", 200000)
]
```

```
dept_df = spark.createDataFrame(dept_data, ["Department", "Budget"])
joined_df = df_clean.join(dept_df, on="Department", how="inner")
joined_df.select("First Name", "Department", "Salary", "Budget").show(5)
```

```
+-----+-----+-----+-----+
|First Name|Department|Salary|Budget|
+-----+-----+-----+-----+
|      Joy|      Legal| 58024|300000|
|    Pedro|      Legal| 55094|300000|
| Michael|      Legal| 76447|300000|
| William|      Legal| 64857|300000|
| Jennifer|      Legal| 72997|300000|
+-----+-----+-----+-----+
only showing top 5 rows
```

```
# --- window function: rank by salary within department ---
dept_window = Window.partitionBy("Department").orderBy(col("Salary").desc())
ranked_df = df_clean.withColumn("salary_rank", rank().over(dept_window))
ranked_df.select("First Name", "Department", "Salary", "salary_rank").show(10)
```

```
+-----+-----+-----+-----+
|First Name|      Department|Salary|salary_rank|
+-----+-----+-----+-----+
|    Jacob|Business Development| 80000|          1|
|   Shelby|Business Development| 80000|          1|
|    Jason|Business Development| 79999|          3|
|   Angela|Business Development| 79999|          3|
|    Jean|Business Development| 79999|          3|
| Matthew|Business Development| 79999|          3|
| Gregory|Business Development| 79999|          3|
| Gabriel|Business Development| 79998|          8|
```

	Lawrence	Business Development	79997	9
	Seth	Business Development	79996	10
+-----+	+-----+			+-----+

only showing top 10 rows

Start coding or [generate](#) with AI.